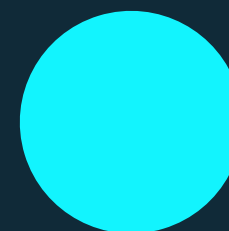
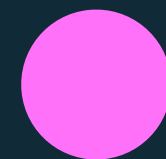




**Não deixe de
registrar a sua
participação!**



FUNDAMENTOS DE VIBE CODE E PROGRAMAÇÃO ASSISTIDA POR IA



1

Vibe Code na prática dev

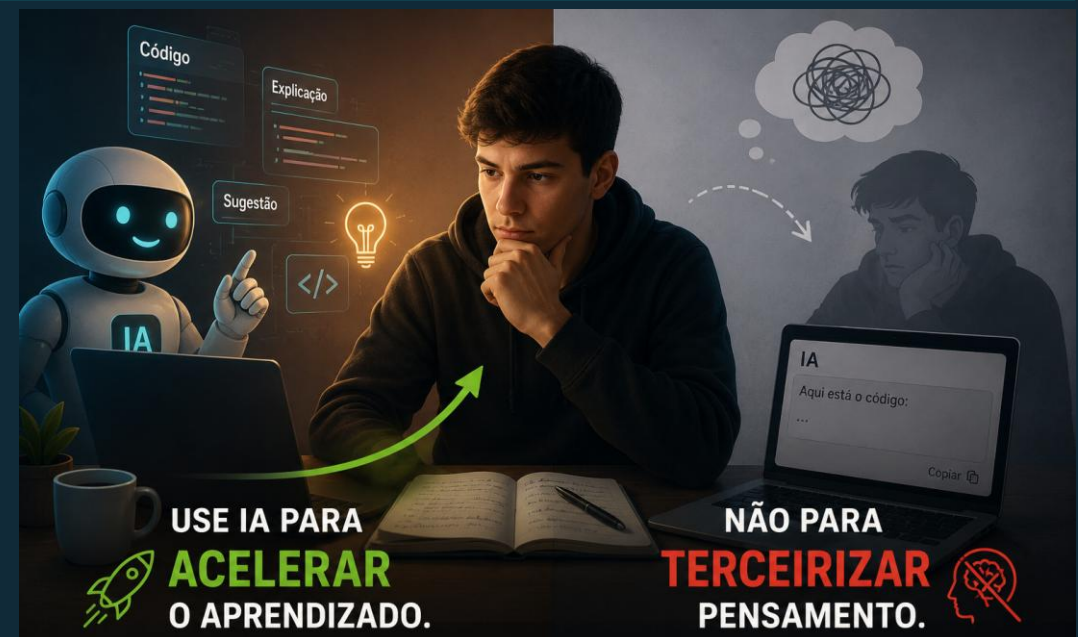


ecossistema de
aprendizagem

Como usar IA no dia a dia sem desligar o cérebro



- Contexto: estagiários de desenvolvimento
- Foco: produtos, catálogo, estoque, pedidos e automações internas

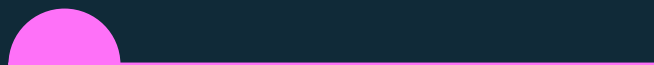


Objetivo da aula

Ao final da aula, você deve conseguir:



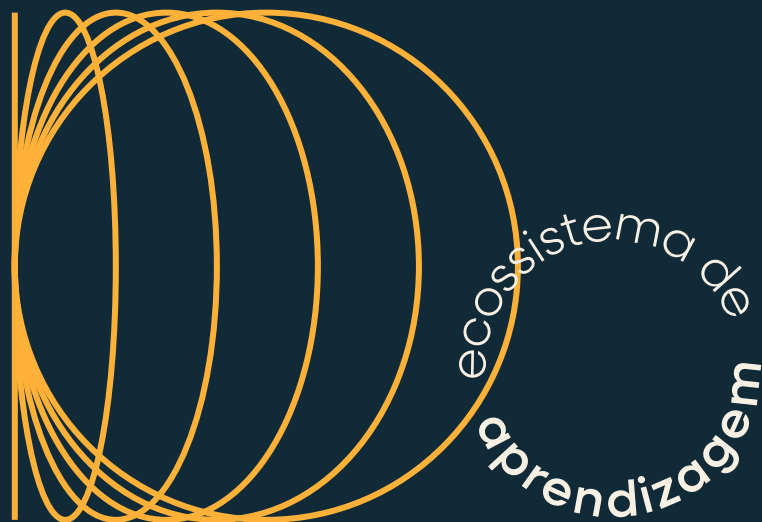
- Entender o que é vite code
- Diferenciar IA, código tradicional e low-code/no-code



- Criar prompts técnicos melhores
- Identificar riscos no código gerado por IA



- Aplicar um ciclo simples: intenção, geração, teste, revisão e documentação



**Conceito de Vibe Code:
programação guiada por IA e
papel do desenvolvedor como
“orquestrador”**

1

Cenário real de trabalho

Onde a IA pode ajudar no dia a dia?



- Entender código legado
- Criar componentes repetitivos
- Gerar testes
- Explicar regras de negócio
- Documentar APIs
- Refatorar funções
- Criar scripts internos
- Revisar mensagens de erro



Exemplo Natura-like:

Criar um componente para exibir um card de produto com nome, preço, imagem, categoria e status de estoque.



DIRETRIZ CENTRAL

A IA sugere.

O dev decide.

O papel do profissional de tecnologia evolui para supervisor de alta performance e arquiteto de qualidade.

ecossistema de
aprendizagem

O que é Vibe Code?

Vibe Code é programar com apoio de IA de forma ágil, utilizando prompts estratégicos em cada etapa:

- **Gerar ideias:** Cocriação acelerada e exploração de abordagens funcionais.
- **Criar código inicial:** Geração instantânea de protótipos e boilerplates.
- **Revisar soluções:** Auditoria veloz de código e detecção de melhorias.
- **Testar hipóteses:** Validação ágil de ideias e soluções em sandbox.
- **Documentar decisões:** Registro automatizado do fluxo lógico adotado.

O papel do desenvolvedor



No Vibe Code, o dev precisa:

- Entender o problema
- Explicar o contexto
- Revisar o código gerado
- Testar o comportamento
- Proteger dados
- Seguir padrões do time
- Documentar o que foi feito



66 Frase-chave

Quem não entende o código, só terceirizou o problema.

Exemplo ruim de prompt

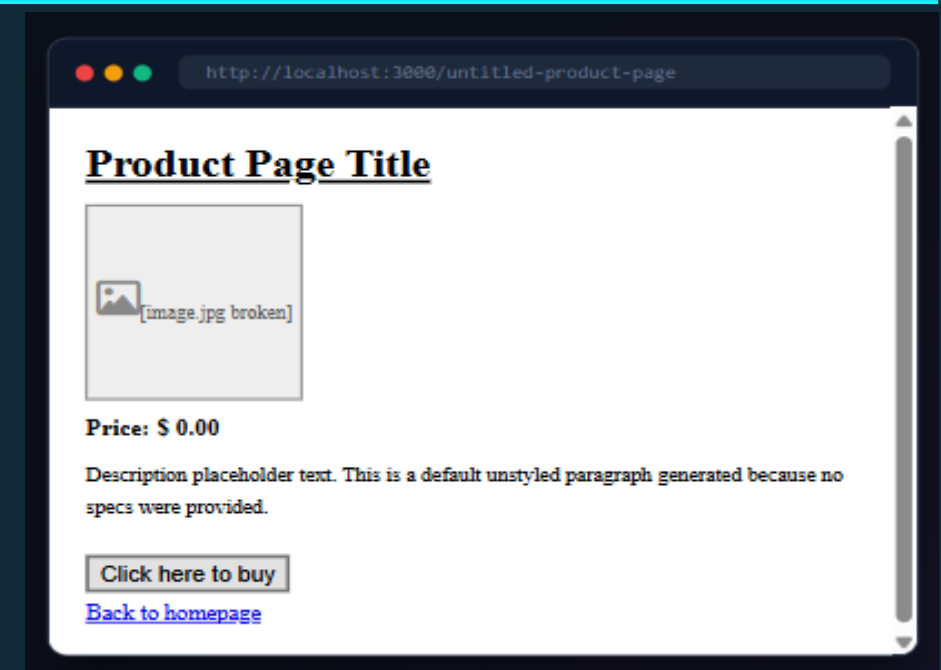


Prompt genérico:

Crie uma tela de produto.

Problemas:

- Não informa tecnologia
- Não diz quais dados existem
- Não define layout
- Não diz se precisa ser responsivo
- Não define regra de estoque



Exemplo melhor de prompt



Crie um componente React com TypeScript para um card de produto.

Dados: id, name, imageUrl, price, category, stockStatus.

Regras:

- Se stockStatus for "unavailable", mostrar "Indisponível"
- Botão desabilitado se indisponível
- CSS simples, sem bibliotecas externas.

Contexto Rico = Código Produzível

Resultado Profissional e Tipado



“



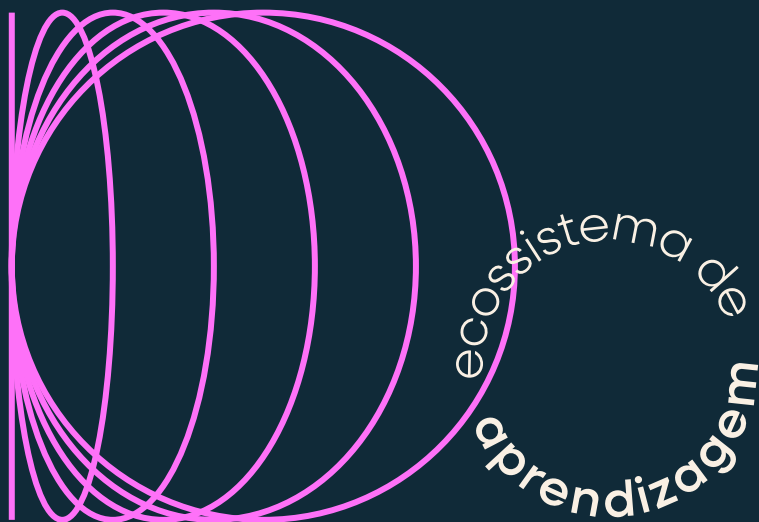
O código está se tornando a commodity.
A clareza de pensamento é o novo luxo.

— A Era do Vibe Coding

”



natura



Diferenças entre código tradicional, low-code/no-code e Vibe Code

2

Código tradicional x low-code/no-code x Vibe Code



Comparação rápida

Abordagem	Como funciona	Ponto forte	Cuidado
Código tradicional	dev escreve tudo	 controle	 mais tempo
Low-code/no-code	ferramenta visual	 rapidez	 limitação
 Vibe Code	dev programa com IA	 velocidade com controle	 revisar sempre

Quando usar cada abordagem



Código Tradicional

Máximo Controle e Robustez

- Regras de negócio críticas
- Arquitetura estrutural
- Segurança e criptografia
- Integrações delicadas



Low-code / No-code

Máxima Velocidade Sem Código

- Validação de protótipos
- Automações simples
- Fluxos de trabalho internos



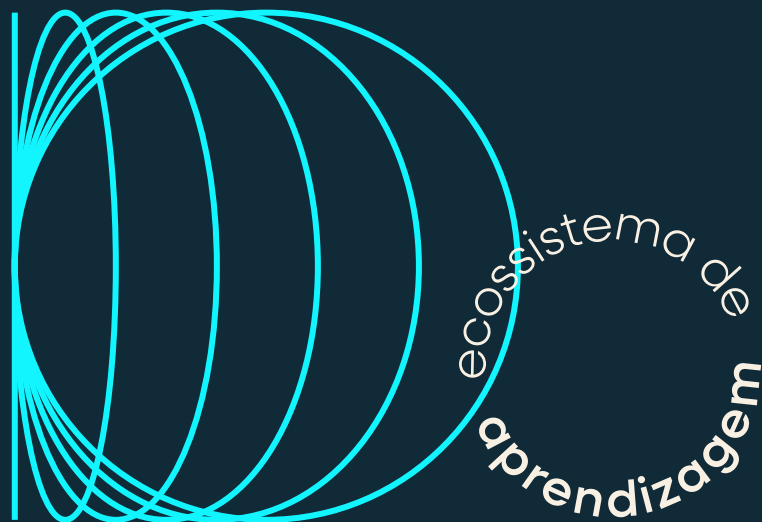
Vibe Code

Simbiose Inteligente e Agilidade

- Criação de componentes
- Geração de testes unitários
- Documentação técnica
- Refatoração e scripts
- MVP e primeira versão



natura



**Boas práticas de uso de IA na
geração de código (limites,
revisão crítica e
responsabilidade técnica)**

3



Boas práticas no uso de IA

Regras de Trabalho:

- Peça uma coisa por vez
- Informe stack e contexto
- Limite o escopo
- Peça explicação
- Revise antes de usar
- Teste antes de commitar
- Compare com padrões do time

Prompt técnico bem estruturado



Modelo

Contexto

Estou em um projeto React com TypeScript.

Objetivo

Criar um componente de card de produto.

Dados disponíveis

name, imageUrl, price, category, stockStatus.

Regras

- botão desabilitado se produto indisponível
- formatar preço em BRL (R\$)
- evitar bibliotecas externas

Saída esperada

Gerar apenas o componente.

UI Card Design





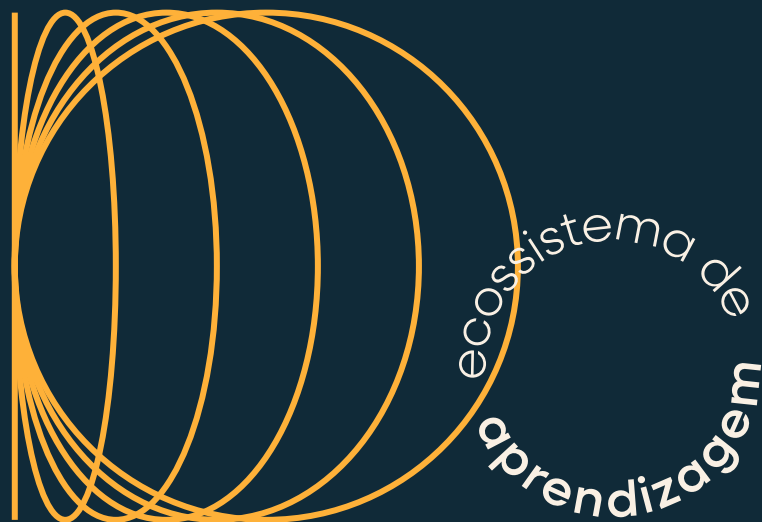
Revisão crítica

Antes de aceitar código da IA, pergunte:

- isso resolve o requisito?
- tem código desnecessário?
- segue o padrão do projeto?
- tem nomes claros?
- tem validação?
- tem risco de segurança?
- tem teste?



o sistema de
aprendizagem



**Riscos de segurança,
privacidade e qualidade ao
utilizar código gerado por IA**



possibilidade de
aprendizagem

Riscos de segurança

Riscos comuns

- Expor tokens
- Vazar dados sensíveis
- Gerar SQL inseguro
- Falhar na autenticação
- Validar dados só no front-end
- Usar dependências desnecessárias

Exemplo de risco



Código inseguro

```
const query = `SELECT * FROM products WHERE name = '${name}'`;
```

Problema

Esse código pode abrir brecha para SQL Injection.



Melhor caminho

```
const query = 'SELECT * FROM products WHERE name = ?';  
db.query(query, [name]);
```




O QUE **NÃO** COLOCAR NO PROMPT

	Dados reais de clientes	×		CPF, e-mail, telefone	×
	Tokens	×		Senhas	×
	Chaves de API	×		Logs internos	×
	...	×			
	Código sensível sem permissão	×		Informações comerciais restritas	×

#SegurançaDaInformação #ProteçãoDeDados

Qualidade do código gerado



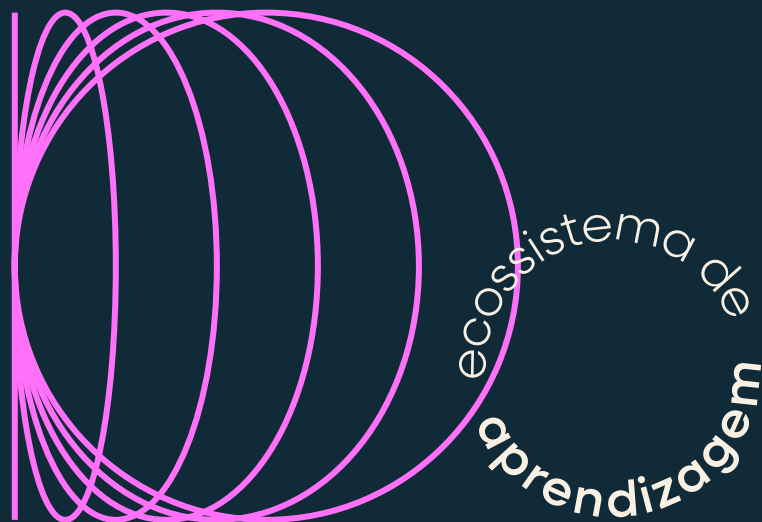
IA pode gerar:

- Duplicação
- Nomes ruins
- Funções grandes
- Validações fracas
- Dependências sem necessidade
- Código difícil de manter
- Teste que não testa nada





natura



**Ciclo de desenvolvimento
orientado a prompts: intenção,
geração, teste, revisão e
documentação**

5

Ciclo orientado a prompts



Exemplo:

Quero criar um componente de produto.

Depois vou testar estoque, preço e estado indisponível.

Em seguida vou revisar nomes, estrutura e documentação.

Etapa 1: intenção



Antes do prompt

Responda:



Qual problema quero resolver?



Quem vai usar?



Quais dados existem?



Quais regras não podem ser quebradas?



Qual tecnologia devo usar?

Exemplo:

- Preciso exibir produtos de uma campanha com preço, imagem e status de estoque.

Etapa 2: geração



Peça uma primeira versão pequena

Foco Atômico

Ao isolar o componente, você garante que a IA não se perca em detalhes de infraestrutura ou boilerplate desnecessário.

PROMPT DE GERAÇÃO

OBJETIVO

Gere apenas o componente ProductCard em React com TypeScript.

RESTRIÇÕES

Não crie API, não crie página, não use biblioteca externa.

Etapa 3: teste



Peça cenários de teste

Liste casos de teste para o componente ProductCard.

Peça cenários de teste

Consideres:

- ☒ **Product AVAILABLE** produto disponível
- ☐ **product UNAVAILABLE** produto indisponível
- ☒ **Warning alert ZERO PRICE** preço zerado
- ☐ **MISSING IMAGE** imagem ausente
- ☐ **EMPTY CATEGORY** categoria vazia

Vibe Coding

Etapa 4: revisão



Prompt de revisão

Critérios de Avaliação:

- **Legibilidade**(Código limpo e intuitivo)
- **Nomes**(Variáveis e funções semânticas)
- **Responsabilidade**(Princípio de Single Resp.)
- **Riscos de Bug**(Edge cases e tipos TS)
- **Acessibilidade**(Tags ARIA e semântica)
- **Melhorias**(Refatoração sem breaking changes)

Exemplo:

PROMPT DE REVISÃO

PROMPT DE REVISÃO

Revise este componente como se fosse um Pull Request.

Peça sugestões de refatoração que mantenham a funcionalidade mas elevem a qualidade técnica.

Etapa 5: documentação



Prompt de documentação: Feche o ciclo com clareza e contexto

O que não pode faltar:

- **Objetivo** (O que o componente resolve?)
- **Props** (Tipagem e obrigatoriedade)
- **Exemplo de uso** (Snippet pronto para copiar)
- **Regras de estoque** (Lógica de exibição/trava)
- **Cuidados conhecidos** (Edge cases e performance)

Dica: Documentação gerada por IA é excelente para manter o padrão do projeto sem gastar tempo manual.

Exemplo:

PROMPT DE DOCUMENTAÇÃO

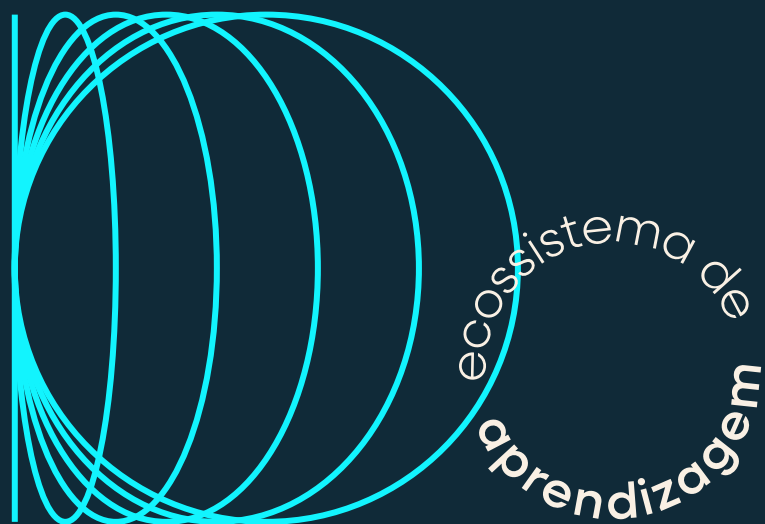
PROMPT

Crie uma documentação curta em Markdown para este componente.

Inclua seções de uso prático e regras de negócio específicas para o estoque.



natura



Prática

6



ecossistema de
aprendizagem

Prática curta: criando uma rule em Markdown

Atividade

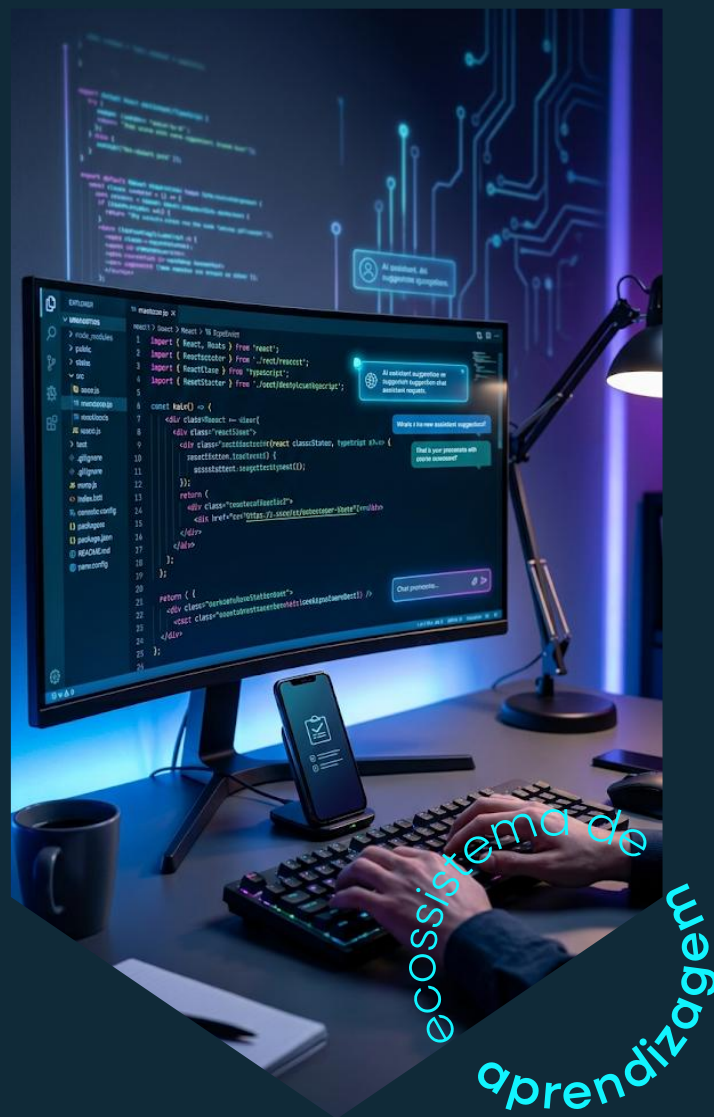
Criar um arquivo:

vibe-product-card.rules.md

Objetivo

Definir regras para usar IA na criação de componentes relacionados a produtos.

Modelo da rule



Rule: criação de componentes de produto

Contexto

Projeto front-end com React e TypeScript para exibição de produtos.



Padrões

- usar componentes pequenos
- usar nomes claros
- evitar bibliotecas externas sem necessidade
- manter responsabilidade única

Dados do produto

- id
- name
- imageUrl
- price
- category
- stockStatus

Regras de negócio

- se stockStatus for "unavailable", desabilitar ação principal
- formatar preço em BRL
- tratar imagem ausente
- evitar expor dados sensíveis

Saída esperada

Gerar código limpo, comentando apenas pontos não óbvios.

Prática: prompt usando a rule



Prompt da atividade

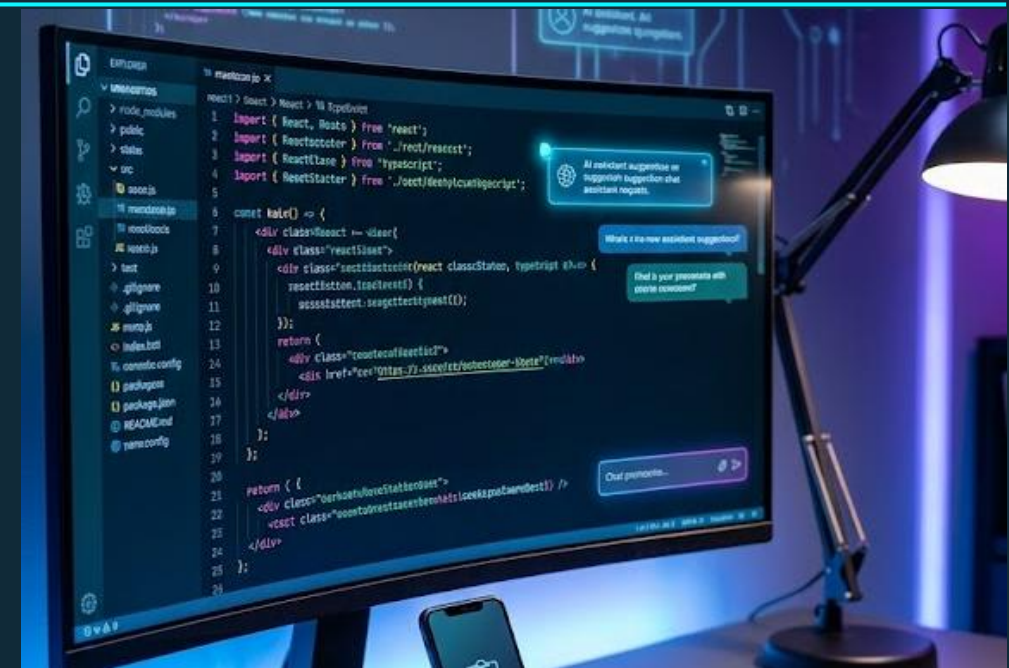
Use as regras abaixo para criar um componente ProductCard.

Produto de exemplo:

```
{
  "id": 1,
  "name": "Hidratante Corporal",
  "imageUrl": "",
  "price": 49.90,
  "category": "Cuidados com o corpo",
  "stockStatus": "available"
}
```

Crie:

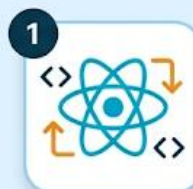
- componente React com TypeScript
- interface Product
- tratamento para imagem ausente
- botão desabilitado se indisponível



Checklist de análise da resposta



Avaliar o código gerado



1

criou props bem definidas?



2

usou TypeScript corretamente?



3

tratou imagem ausente?



5

respeitou **produto indisponível?**



6

evitou **dependências desnecessárias?**



7

ficou fácil de **manter?**



```
export default function() {  
  return () =>  
    <propos at  
    <button 'Comprar' />  
}
```

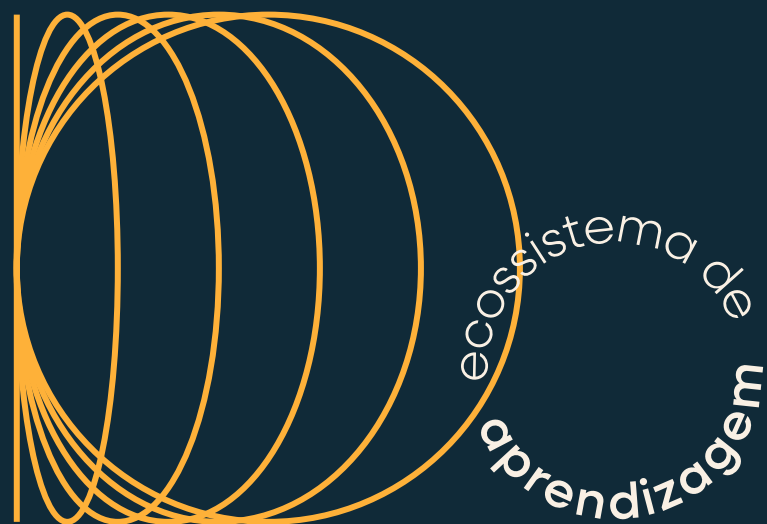



Prompt de melhoria

Ajuste iterativo

O código ficou bom, mas ajuste os pontos abaixo:

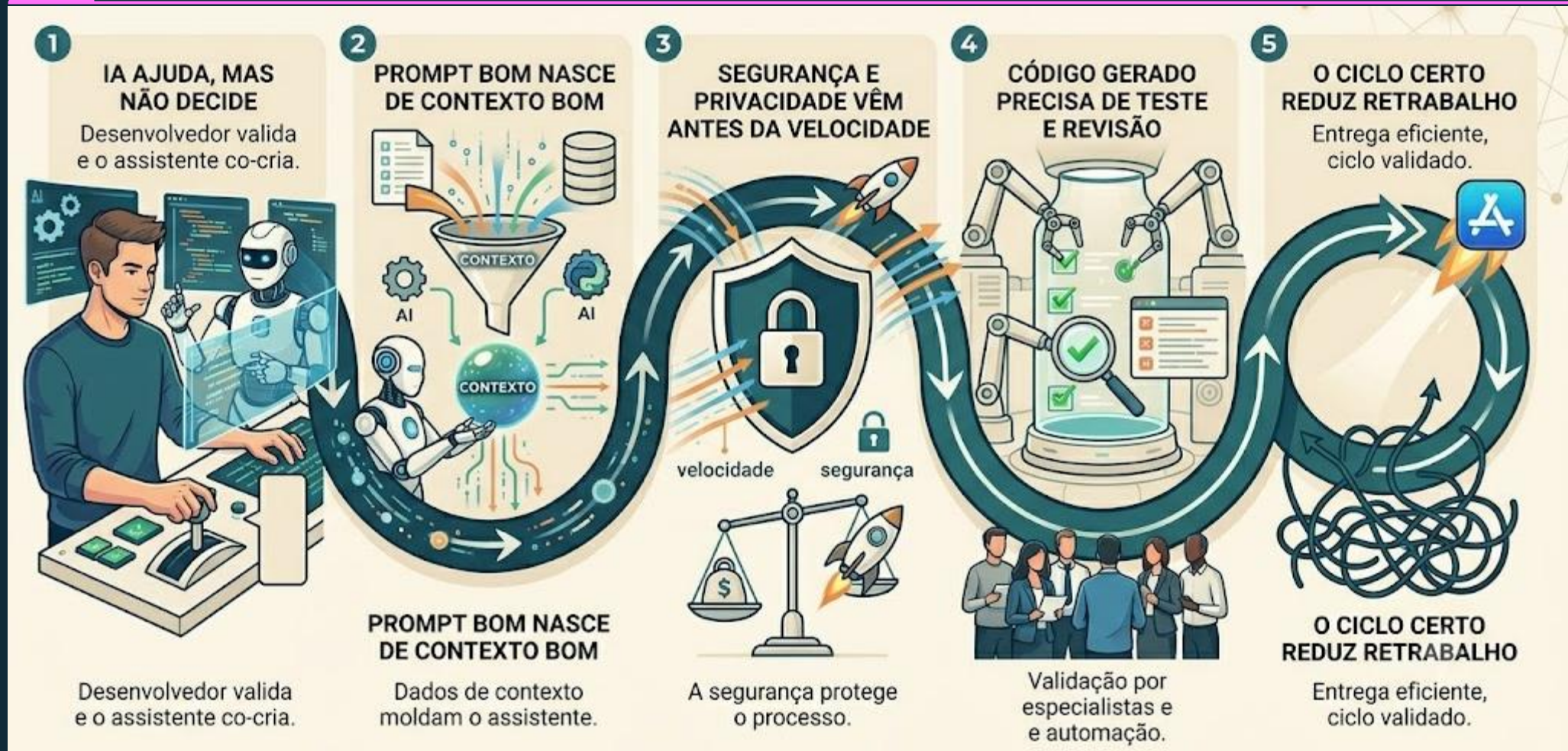
- separar a função de formatação de preço
- melhorar nomes das classes
- adicionar aria-label no botão
- remover comentários óbvios
- manter o mesmo comportamento



Fechamento

7

O que aprendemos?



“



Use IA para acelerar o aprendizado.
Não para terceirizar pensamento.

Elaboração própria do autor

”

Próximo encontro



1. Escolha e uso de ferramentas de Vibe Code (assistentes de código, IDEs e agentes de IA)
2. Escrita de prompts técnicos para gerar funções, APIs simples e automações internas
3. Estratégias de teste: casos de uso, validação de saída e ajustes iterativos com IA
4. Refatoração e documentação de soluções geradas por IA para trabalho em equipe

**Não deixe de
avaliar o nosso
encontro!**

